

Short Template:

```
#include <bits/stdc++.h>
using namespace std;
typedef long long int lli;
#define endl "\n"
void fast() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
}
void soln() {}
int main() {
    fast();
    lli t = 1;
    cin >> t;
    while (t--) soln();
}
// Author: tashobi_02 //
```

Binary Search:

```
lli binary_search(vector<lli> vec, lli lo, lli hi, lli x) {
    while (lo <= hi) {
        lli mid = lo + (hi - lo) / 2;
        if (vec[mid] == x)
            return mid;
        else if (vec[mid] < x)
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    return -1;
}

lli lower_bound(vector<lli>& v, lli e) {
    lli lo = 0, hi = v.size() - 1;
    lli mid;
    while (lo <= hi) {
        lli mid = lo + (hi - lo) / 2;
        if (v[mid] < e)
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    if (v[lo] >= e)
        return lo;
    if (v[hi] >= e)
        return hi;
    return -1;
}

lli upper_bound(vector<lli>& v, lli e) {
    lli lo = 0, hi = v.size() - 1;
    lli mid;
    while (lo <= hi) {
        lli mid = lo + (hi - lo) / 2;
        if (v[mid] <= e)
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    if (v[lo] >= e)
        return lo;
    if (v[hi] >= e)
        return hi;
    return -1;
}

void soln() {
    lli n;
    cin >> n;
    vector<lli> vec(n);
    for (lli i = 0; i < n; i++) cin >> vec[i];
```

Binary Search Tree:

```
struct Node {
    lli key;
    Node* left;
    Node* right;
    Node(lli item) {
        key = item;
        left = NULL;
        right = NULL;
    }
};

Node* insert(Node* node, lli key) {
    if (node == NULL) {
        return new Node(key);
    }
    if (node->key == key) {
        return node;
    }
    if (node->key < key) {
        node->right = insert(node->right, key);
    } else {
        node->left = insert(node->left, key);
    }
    return node;
}

Node* search(Node* root, lli key) {
    if (root == NULL || root->key == key) {
        return root;
    }
    if (root->key < key) {
        return search(root->right, key);
    }
    return search(root->left, key);
}

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->key << " ";
        inorder(root->right);
    }
}

void preorder(Node* root) {
    if (root != NULL) {
        cout << root->key << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->key << " ";
    }
}

void levelorder(Node* root) {
    queue<Node*> Q;
    Q.push(root);
    while (!Q.empty()) {
        Node* current = Q.front();
        Q.pop();
        cout << current->key << " ";
        if (current->left != NULL)
            Q.push(current->left);
        if (current->right != NULL)
            Q.push(current->right);
    }
}
```

```

lli x;
cin >> x;
lli y = binary_search(vec, 0, n - 1, x);
cout << y << endl;
}

```

Prefix Sum:

```

void soln() {
    lli n, queries;
    cin >> n >> queries;
    vector<lli> arr(n);
    for (lli i = 0; i < n; i++) cin >> arr[i];
    vector<lli> pfs(n);
    pfs[0] = arr[0];
    for (lli i = 0; i < n - 1; i++) {
        pfs[i + 1] = pfs[i] + arr[i + 1];
    }
    // for (lli i = 0; i < n; i++) cout << pfs[i] << "
";
    // cout << endl;
    while (queries--) { // O(q)
        lli a, b;
        cin >> a >> b;
        cout << pfs[b - 1] - pfs[a - 1] + arr[a - 1] <<
endl; // O(1)
    }
}

```

Prefix Sum 2D:

```

void soln() {
    lli n, m, queries;
    cin >> n >> m >> queries;
    vector<vector<lli>> grid(n, vector<lli>(m));
    for (lli i = 0; i < n; i++)
        for (lli j = 0; j < m; j++) cin >> grid[i][j];
    vector<vector<lli>> gridSum(n, vector<lli>(m));
    gridSum[0][0] = grid[0][0];
    // Updating only the first row and column
    for (lli i = 1; i < m; i++) gridSum[0][i] =
gridSum[0][i - 1] + grid[0][i];
    for (lli i = 1; i < n; i++) gridSum[i][0] =
gridSum[i - 1][0] + grid[i][0];
    for (lli i = 1; i < n; i++) {
        for (lli j = 1; j < m; j++) {
            gridSum[i][j] = gridSum[i - 1][j] +
gridSum[i][j - 1] -
gridSum[i - 1][j - 1] +
grid[i][j];
        }
    }
    // for (lli i = 0; i < n; i++) {
    //     for (lli j = 0; j < m; j++) {
    //         cout << gridSum[i][j] << " ";
    //     }
    //     cout << endl;
    // }
    while (queries--) {
        lli x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        // Convert 1-based to 0-based indexing
        x1--, y1--, x2--, y2--;
        lli ans = gridSum[x2][y2];
        if (x1 > 0)
            ans -= gridSum[x1 - 1][y2];
        if (y1 > 0)
            ans -= gridSum[x2][y1 - 1];
        if (x1 > 0 && y1 > 0)
            ans += gridSum[x1 - 1][y1 - 1];
        cout << ans << endl;
    }
}

```

```

lli findHeightDiameter(Node* node, lli& diameter) {
    if (node == NULL)
        return 0;
    lli lh = findHeightDiameter(node->left, diameter);
    lli rh = findHeightDiameter(node->right, diameter);

    diameter = max(diameter, lh + rh);

    return max(lh, rh) + 1;
}

lli diameterBST(Node* node) {
    lli diameter = 0;
    findHeightDiameter(node, diameter);
    return diameter;
}

lli maxPathDown(Node* node, lli& maxi) {
    if (node == NULL)
        return 0;
    lli l = max(0LL, maxPathDown(node->left, maxi));
    lli r = max(0LL, maxPathDown(node->right, maxi));
    maxi = max(maxi, l + r + node->key);
    return max(l, r) + node->key;
}

lli maxPathSum(Node* node) {
    lli maxi = LLONG_MIN;
    maxPathDown(node, maxi);
    return maxi;
}

void soln() {
    lli n, x;
    Node* root;
    cin >> n;
    for (lli i = 0; i < n; i++) {
        cin >> x;
        if (i == 0) {
            root = new Node(x);
        } else {
            insert(root, x);
        }
    }
    cout << "Inorder: ";
    inorder(root);
    cout << endl << "Preorder: ";
    preorder(root);
    cout << endl << "Postorder: ";
    postorder(root);
    cout << endl << "Levelorder: ";
    levelorder(root);
    lli diameter = 0;
    cout << endl << "Height of the tree: " <<
findHeightDiameter(root, diameter)
<< endl;
    cout << "Diameter of the tree: " <<
diameterBST(root) << endl;
    cout << "Max Path Sum: " << maxPathSum(root) << endl;
    lli ss;
    cin >> ss;
    (search(root, ss) != NULL) ? cout << "Found" : cout
<< "Not Found";
    cout << endl;
}

```

Binary Search Tree Deletion:

```

struct Node {
    lli key;
    Node* left;
    Node* right;
    Node* parent;
    Node(lli item) {

```

```

    }
}

```

Prefix XOR:

```

void soln() {
    lli n, queries;
    cin >> n >> queries;
    vector<lli> arr(n);
    for (lli i = 0; i < n; i++) cin >> arr[i];
    vector<lli> xorSum(n);
    xorSum[0] = arr[0];
    for (lli i = 0; i < n - 1; i++) xorSum[i + 1] =
xorSum[i] ^ arr[i + 1];
    // for (lli i = 0; i < n; i++) cout << xorSum[i] <<
    " ";
    // cout << endl;
    while (queries--) {
        lli a, b;
        cin >> a >> b;
        cout << (xorSum[b - 1] ^ xorSum[a - 1] ^ arr[a -
1]) << endl;
    }
}

```

Difference Array:

```

/* Given an array of N elements perform Q queries on it
and print the final
array after performing all the queries. N <= 1e5, Q <=
1e5.
Every query is of the form Li, Ri, Xi.
The array is initialized to all zeroes. Add Xi form Li
to Ri-th index and then
print the final array */
// Brute force soln is O(N*Q) which is inefficient
void soln() {
    lli n, queries;
    cin >> n >> queries;
    vector<lli> arr(n, 0);
    while (queries--) {
        lli l, r, x;
        cin >> l >> r >> x;
        // l--, r--; if 1 based index
        arr[l] += x;
        if (r + 1 < n)
            arr[r + 1] -= x;
    }
    vector<lli> pfs(n);
    pfs[0] = arr[0];
    for (lli i = 1; i < n; i++) pfs[i] = pfs[i - 1] +
arr[i];
    for (lli i = 0; i < n; i++) cout << pfs[i] << " ";
    cout << endl;
}
/* Given an array of N elements perform Q queries on it
and print the final
array after performing all the queries. N <= 1e5, Q <=
1e5.
Every query is of the form Li, Ri, Xi.
The array is initialized to all zeroes. Add Xi form Li
to Ri-th index and then
print the final array */
// Brute force soln is O(N*Q) which is inefficient
void soln() {
    lli n, queries;
    cin >> n >> queries;
    vector<lli> arr(n, 0);
    while (queries--) {
        lli l, r, x;
        cin >> l >> r >> x;
        // l--, r--; if 1 based index

```

```

        key = item;
        left = NULL;
        right = NULL;
        parent = NULL;
    }
};

Node* insert(Node* node, lli key) {
    if (node == NULL) {
        return new Node(key);
    }
    if (node->key == key) {
        return node;
    }
    if (node->key < key) {
        Node* rightChild = insert(node->right, key);
        node->right = rightChild;
        rightChild->parent = node;
    } else {
        Node* leftChild = insert(node->left, key);
        node->left = leftChild;
        leftChild->parent = node;
    }
    return node;
}

Node* search(Node* root, lli key) {
    if (root == NULL || root->key == key) {
        return root;
    }
    if (root->key < key) {
        return search(root->right, key);
    }
    return search(root->left, key);
}

Node* minimum(Node* node) {
    while (node->left != NULL) {
        node = node->left;
    }
    return node;
}

void transplant(Node*& root, Node* u, Node* v) {
    if (u->parent == NULL) {
        root = v;
    } else if (u == u->parent->left) {
        u->parent->left = v;
    } else {
        u->parent->right = v;
    }
    if (v != NULL) {
        v->parent = u->parent;
    }
}

Node* tree_delete(Node*& root, lli key) {
    Node* z = search(root, key);
    if (z == NULL)
        return root;
    if (z->left == NULL) {
        transplant(root, z, z->right);
    } else if (z->right == NULL) {
        transplant(root, z, z->left);
    } else {
        Node* y = minimum(z->right);
        if (y->parent != z) {
            transplant(root, y, y->right);
            y->right = z->right;
            y->right->parent = y;
        }
        transplant(root, z, y);
        y->left = z->left;
        y->left->parent = y;
    }
    delete z;
}

```

```

arr[l] += x;
if (r + 1 < n)
    arr[r + 1] -= x;
}
vector<lli> pfs(n);
pfs[0] = arr[0];
for (lli i = 1; i < n; i++) pfs[i] = pfs[i - 1] +
arr[i];
for (lli i = 0; i < n; i++) cout << pfs[i] << " ";
cout << endl;
}

```

Most Frequent Character in a Substring:

```

void soln() {
    string s;
    lli q;
    cin >> s >> q;
    const lli Alpha = 26;
    vector<vector<lli>> pfsFreq;
    lli n = s.size();
    pfsFreq.assign(n + 1, vector<lli>(Alpha, 0));
    for (lli i = 1; i <= n; i++) {
        pfsFreq[i] = pfsFreq[i - 1];
        pfsFreq[i][s[i - 1] - 'a']++;
    }
    while (q--) {
        lli l, r;
        cin >> l >> r;
        l--, r--;
        lli mx = 0;
        char mostCommon = 'a';
        for (lli i = 0; i < Alpha; i++) {
            lli ct = pfsFreq[r + 1][i] - pfsFreq[l][i];
            if (ct > mx) {
                mx = ct;
                mostCommon = 'a' + i;
            }
        }
        cout << mostCommon << endl;
    }
}

```

Sparse Table:

```

const lli N = 2e6 + 10;
const lli M = 21; // log2(N) ≈ 21
vector<vector<lli>> table(N, vector<lli>(M, 0));
void buildSparseTable(vector<lli>& arr, lli n) {
    for (lli i = 0; i < n; i++) table[i][0] = arr[i];
    for (lli j = 1; (1 << j) <= n; j++) {
        for (lli i = 0; (i + (1 << j) - 1) < n; i++) {
            table[i][j] =
                min(table[i][j - 1], table[i + (1 << (j
- 1))][j - 1]);
        }
    }
}
void soln() {
    lli n, q;
    cin >> n >> q;
    vector<lli> arr(n);
    for (lli i = 0; i < n; i++) cin >> arr[i];
    buildSparseTable(arr, n);
    while (q--) {
        lli l, r;
        cin >> l >> r;
        l--, r--; // Convert to 0-based indexing
        lli p = (lli)log2(r - l + 1);
        lli k = (1 << p);
        lli ans = min(table[l][p], table[r - k + 1][p]);
    }
}

```

```

return root;
}

```

```

void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        cout << root->key << " ";
        inorder(root->right);
    }
}

```

```

void preorder(Node* root) {
    if (root != NULL) {
        cout << root->key << " ";
        preorder(root->left);
        preorder(root->right);
    }
}

```

```

void postorder(Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        cout << root->key << " ";
    }
}

```

```

void levelorder(Node* root) {
    if (root == NULL)
        return;
    queue<Node*> Q;
    Q.push(root);
    while (!Q.empty()) {
        Node* current = Q.front();
        Q.pop();
        cout << current->key << " ";
        if (current->left != NULL)
            Q.push(current->left);
        if (current->right != NULL)
            Q.push(current->right);
    }
}

```

```

lli findHeight(Node* node) {
    lli lh, rh;
    if (node == NULL)
        return -1;
    else {
        lh = findHeight(node->left);
        rh = findHeight(node->right);
    }
    return max(lh, rh) + 1;
}

```

```

void soln() {
    lli n, x;
    Node* root = NULL;
    cin >> n;
    for (lli i = 0; i < n; i++) {
        cin >> x;
        if (i == 0) {
            root = new Node(x);
        } else {
            insert(root, x);
        }
    }
    cout << "Inorder: ";
    inorder(root);
    cout << endl << "Preorder: ";
    preorder(root);
    cout << endl << "Postorder: ";
    postorder(root);
    cout << endl << "Levelorder: ";
    levelorder(root);
    cout << endl << "Height of the tree: " <<
    findHeight(root) << endl;
    lli deleteKey;
}

```

```

    cout << ans << endl;
}
}

```

Trie:

```

struct Node {
    Node* links[26];
    bool flag = false;

    bool containsKey(char ch) { return (links[ch - 'a'])
    != NULL; }
    void put(char ch, Node* node) { links[ch - 'a'] =
    node; }
    Node* get(char ch) { return links[ch - 'a']; }
    void endMark() { flag = true; }
    bool isEnd() { return flag; }
};

class Trie {
private:
    Node* root;

public:
    Trie() { root = new Node(); }

    void insert(string word) {
        Node* node = root;
        for (int i = 0; i < (int)word.length(); i++) {
            if (!node->containsKey(word[i])) {
                node->put(word[i], new Node());
            }
            node = node->get(word[i]);
        }
        node->endMark();
    }

    bool search(string word) {
        Node* node = root;
        for (int i = 0; i < (int)word.length(); i++) {
            if (!node->containsKey(word[i])) {
                return false;
            }
            node = node->get(word[i]);
        }
        return node->isEnd();
    }

    bool startsWith(string prefix) {
        Node* node = root;
        for (int i = 0; i < (int)prefix.length(); i++) {
            if (!node->containsKey(prefix[i])) {
                return false;
            }
            node = node->get(prefix[i]);
        }
        return true;
    }

    void display(Node* curr, string prefix) {
        if (curr->isEnd()) {
            cout << prefix << " ";
        }
        for (int i = 0; i < 26; i++) {
            if (curr->links[i] != NULL) {
                char nextChar = i + 'a';
                display(curr->links[i], prefix +
                nextChar);
            }
        }
    }

    void displayAll() { display(root, ""); }
};

```

```

    cout << "Enter key to delete: ";
    cout.flush();
    cin >> deleteKey;
    root = tree_delete(root, deleteKey);
    cout << "Inorder after deletion: ";
    inorder(root);
    cout << endl;
}

```

AVL Tree:

```

struct Node {
    lli key;
    Node *left;
    Node *right;
    lli height;
    Node(lli k) {
        key = k;
        left = nullptr;
        right = nullptr;
        height = 1;
    }
};

lli height(Node *N) {
    if (N == nullptr)
        return 0;
    return N->height;
}

Node *rightRotate(Node *y) {
    Node *x = y->left;
    Node *temp = x->right;
    // Starts rotation
    x->right = y;
    y->left = temp;
    // Save heights
    y->height = 1 + max(height(y->left), height(y-
    >right));
    x->height = 1 + max(height(x->left), height(x-
    >right));
    return x;
}

Node *leftRotate(Node *x) {
    Node *y = x->right;
    Node *temp = y->left;
    // Starts rotation
    y->left = x;
    x->right = temp;
    // Save heights
    x->height = 1 + max(height(x->left), height(x-
    >right));
    y->height = 1 + max(height(y->left), height(y-
    >right));
    return y;
}

lli getBalance(Node *N) {
    if (N == nullptr)
        return 0;
    return height(N->left) - height(N->right);
}

Node *insert(Node *node, lli key) {
    if (node == nullptr)
        return new Node(key);
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);
    else
        return node;
    node->height = 1 + max(height(node->left),
    height(node->right));
    lli balance = getBalance(node);
    if (balance > 1 && key < node->left->key) // LL

```

```

void soln() {
    Trie trie;
    string line;

    getline(cin, line);
    stringstream ss1(line);
    string word;
    while (ss1 >> word) {
        trie.insert(word);
    }
    trie.displayAll();
    cout << endl;
    getline(cin, line);
    stringstream ss2(line);
    while (ss2 >> word) {
        cout << (trie.search(word) ? "T" : "F") << " ";
    }
    cout << endl;
}

```

Trie Max XOR Pair:

```

struct Node {
    Node* links[2];
    Node() { links[0] = links[1] = nullptr; }
    bool containsKey(int bit) { return links[bit] != nullptr; }
    void put(int bit, Node* node) { links[bit] = node; }
    Node* get(int bit) { return links[bit]; }
};

class Trie {
private:
    Node* root;

public:
    Trie() { root = new Node(); }

    void insert(int num) {
        Node* node = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (!node->containsKey(bit)) {
                node->put(bit, new Node());
            }
            node = node->get(bit);
        }
    }

    int getMax(int num) {
        Node* node = root;
        int maxNum = 0;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (node->containsKey(1 - bit)) {
                maxNum |= (1 << i);
                node = node->get(1 - bit);
            } else {
                node = node->get(bit);
            }
        }
        return maxNum;
    }
};

int maxXOR(vector<int>& arr) {
    Trie trie;
    for (auto& it : arr) {
        trie.insert(it);
    }
    int maxI = 0;
    for (auto& it : arr) {
        maxI = max(maxI, trie.getMax(it));
    }
}

```

```

return rightRotate(node);
if (balance < -1 && key > node->right->key) // RR
    return leftRotate(node);
if (balance > 1 && key > node->left->key) { // LR
    node->left = leftRotate(node->left);
    return rightRotate(node);
}
if (balance < -1 && key < node->right->key) { // RL
    node->right = rightRotate(node->right);
    return leftRotate(node);
}
return node;
}

void preOrder(Node *root) {
    if (root != nullptr) {
        cout << root->key << " ";
        preOrder(root->left);
        preOrder(root->right);
    }
}

void soln() {
    Node *root = nullptr;
    root = insert(root, 50);
    root = insert(root, 40);
    root = insert(root, 35);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 10);
    root = insert(root, 5);
    preOrder(root);
    return;
}

```

Binary Index Tree/ Fenwick Tree:

```

lli getSum(vector<lli>& BITree, lli ix) {
    lli sum = 0;
    ix++;
    while (ix > 0) {
        sum += BITree[ix];
        ix -= ix & (-ix);
    }
    return sum;
}

void updateBIT(vector<lli>& BITree, lli n, lli ix, lli val) {
    ix++;
    while (ix <= n) {
        BITree[ix] += val;
        ix += ix & (-ix);
    }
}

vector<lli> constructBITree(vector<lli>& arr, lli n) {
    vector<lli> BITree(n + 1, 0);
    for (lli i = 0; i < n; i++) updateBIT(BITree, n, i, arr[i]);
    return BITree;
}

void soln() {
    lli n, q;
    cin >> n >> q;
    vector<lli> arr(n);
    for (lli i = 0; i < n; i++) cin >> arr[i];
    vector<lli> BITree = constructBITree(arr, n);
    while (q--) {
        lli f;
        cin >> f; // f = 1 -> update value, f = 2 ->
        getSum of a range
        if (f == 1) {
            lli ix, x;
            cin >> ix >> x;
            ix--; // if 1 based
            lli diff = x - arr[ix];
            arr[ix] = x;
        }
    }
}

```

```

    }
    return maxI;
}

void soln() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << maxXOR(arr) << endl;
}

struct Node {
    Node* links[2];
    Node() { links[0] = links[1] = nullptr; }
    bool containsKey(int bit) { return links[bit] !=
        nullptr; }
    void put(int bit, Node* node) { links[bit] = node; }
    Node* get(int bit) { return links[bit]; }
};

class Trie {
private:
    Node* root;

public:
    Trie() { root = new Node(); }

    void insert(int num) {
        Node* node = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (!node->containsKey(bit)) {
                node->put(bit, new Node());
            }
            node = node->get(bit);
        }
    }

    int getMax(int num) {
        Node* node = root;
        int maxNum = 0;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (node->containsKey(1 - bit)) {
                maxNum |= (1 << i);
                node = node->get(1 - bit);
            } else {
                node = node->get(bit);
            }
        }
        return maxNum;
    }
};

int maxXOR(vector<int>& arr) {
    Trie trie;
    for (auto& it : arr) {
        trie.insert(it);
    }
    int maxI = 0;
    for (auto& it : arr) {
        maxI = max(maxI, trie.getMax(it));
    }
    return maxI;
}

void soln() {
    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    cout << maxXOR(arr) << endl;
}

```

```

        updateBIT(BITree, n, ix, diff);
    } else if (f == 2) {
        lli l, r;
        cin >> l >> r;
        l--, r--; // if 1 based
        lli ans = getSum(BITree, r) - getSum(BITree,
            l - 1);
        cout << ans << endl;
    }
}
}

```

Disjoint Set Union:

```

const lli N = 1e5 + 10;
lli parent[N];
lli rank_[N];
void make(lli v) {
    parent[v] = v;
    rank_[v] = 0;
}
lli find(lli v) {
    if (v == parent[v])
        return v;
    return parent[v] = find(parent[v]); // Path
    Compression
}
void Union(lli a, lli b) {
    a = find(a);
    b = find(b);
    if (a != b) {
        if (rank_[a] <= rank_[b]) {
            parent[a] = b;
        } else {
            parent[b] = a;
        }
        if (rank_[a] == rank_[b]) {
            rank_[b]++;
        }
    }
}
void print_parent(lli n) {
    for (lli i = 0; i < n; i++) {
        cout << parent[i] << "(" << rank_[i] << ") ";
    }
    cout << endl;
}
int main() {
    fast();
    lli n;
    cin >> n;
    for (lli i = 0; i < n; i++) {
        make(i);
    }
    while (true) {
        lli type;
        cin >> type;
        if (type == 1) {
            print_parent(n);
        } else if (type == 2) {
            lli x;
            cin >> x;
            cout << find(x) << endl;
        } else if (type == 3) {
            lli u, v;
            cin >> u >> v;
            Union(u, v);
            print_parent(n);
        } else if (type == 4) {
            print_parent(n);
        } else {
            break;
        }
    }
}

```


Hashing Open Addressing:

```
class HashTable {
private:
    vector<lli> table;
    lli size, ct, cc;

    lli hash1(lli x) { return x % size; }
    lli hash2(lli x) { return 7 - (x % 7); }

    lli probe(lli x, lli i) {
        if (cc == 1)
            return (hash1(x) + i) % size;
        if (cc == 2)
            return (hash1(x) + i * i) % size;
        if (cc == 3)
            return (hash1(x) + i * hash2(x)) % size;
        return -1;
    }

public:
    HashTable(lli cc, lli sz) : cc(cc), size(sz), ct(0)
    {
        table.assign(sz, -1);
    }

    void insert(lli x) {
        lli i = 0;
        while (i < 6) {
            lli index = probe(x, i);
            if (table[index] == -1) {
                table[index] = x;
                ct++;
                cout << "Inserted : Index - " << index
                    << " (L.F = " << fixed
                        << setprecision(3) << (double)ct /
                            size << ")" << endl;
                return;
            } else {
                cout << "Collision: Index - " << index
                    << endl;
            }
            i++;
        }
        cout << "Input Abandoned" << endl;
    }
};
```

Hashing Separate Chaining:

```
class HashTable {
private:
    lli size, itemCount = 0;
    vector<list<lli>> table;

    lli hashFunction(lli key) { return key % size; }

public:
    HashTable(lli s) : size(s), table(s) {}

    void insert(lli key) {
        table[hashFunction(key)].push_front(key);
        itemCount++;
    }

    bool search(lli key) {
        for (lli num : table[hashFunction(key)]) {
            if (num == key)
                return true;
        }
    }
};
```

```
return 0;
}
```

Rabin Karp String Matching Algorithm:

```
const lli BASE = 99991; // Large Prime Number
const lli M = 1e9 + 9;

// Implementation of Rabin-Karp Algorithm
lli RK(const string& text, const string& pattern) {
    lli T = text.size();
    lli P = pattern.size();
    // Precomputing powers of BASE modulo M
    vector<lli> powers(max(T, P));
    powers[0] = 1;
    for (lli i = 1; i < (lli)powers.size(); i++) {
        powers[i] = (powers[i - 1] * BASE) % M;
    }
    // for (auto px : powers) {
    //     cout << px << " ";
    // }
    // cout << endl;
    // Computing prefix hashes for the text
    vector<lli> hash(T + 1, 0);
    for (lli i = 0; i < T; i++) {
        hash[i + 1] = (hash[i] + (text[i] - 'a' + 1) *
            powers[i]) % M;
    }
    // for (auto hh : hash) {
    //     cout << hh << " ";
    // }
    // cout << endl;
    // Computing hash of the pattern
    lli pattern_hash = 0;
    for (lli i = 0; i < P; i++) {
        pattern_hash = (pattern_hash + (pattern[i] - 'a'
            + 1) * powers[i]) % M;
    }
    // cout << pattern_hash << endl;
    // Counting occurrences of the pattern in the text
    lli count = 0;
    for (lli i = 0; i + P - 1 < T; i++) {
        lli current_hash = (hash[i + P] + M - hash[i]) %
            M;
        // cout << current_hash << endl;
        if (current_hash == pattern_hash * powers[i] %
            M) {
            count++;
        }
    }
    return count;
}

void soln() {
    string text, pattern;
    cin >> text >> pattern;
    lli ans = RK(text, pattern);
    cout << ans << endl;
}
```

String Matching Using KMP Algorithm:

```
vector<lli> pi_array(string pattern) {
    lli n = pattern.length();
    vector<lli> pi(n);
    pi[0] = 0;
    for (lli i = 1; i < n; i++) {
        pi[i] = 0;
        lli j = pi[i - 1];
        while (j > 0 && pattern[j] != pattern[i]) {
            j = pi[j - 1];
        }
    }
}
```

```

        return false;
    }

    void remove(lli key) {
        table[hashFunction(key)].remove(key);
        itemCount--;
    }

    double getLoadFactor() { return (double)itemCount /
size; }

    lli getMaxLoadFactor() {
        lli maxLen = 0;
        for (const auto& lst : table) {
            maxLen = max(maxLen, (lli)lst.size());
        }
        return maxLen;
    }

    void display() {
        for (lli i = 0; i < size; i++) {
            cout << i << ": ";
            for (lli num : table[i]) {
                cout << num << " -> ";
            }
            cout << "NULL" << endl;
        }
        cout << "Load Factor: " << fixed <<
setprecision(2) << getLoadFactor()
        << endl;
        cout << "Maximum Chain Length: " <<
getMaxLoadFactor() << endl;
    }
};

```

BFS:

```

const lli N = 1e7 + 10;
vector<bool> visited(N);
void BFS(lli start, vector<vector<lli>>& graph) {
    queue<lli> q;
    q.push(start);
    visited[start] = true;
    while (!q.empty()) {
        lli v = q.front();
        cout << v << " ";
        q.pop();
        for (auto i = graph[v].begin(); i !=
graph[v].end(); ++i) {
            if (!visited[*i]) {
                q.push(*i);
                visited[*i] = true;
            }
        }
    }
}

int main() {
    fast();
    lli n, e;
    cin >> n >> e;
    vector<vector<lli>> graph(n + 1);
    lli nodes, neighbor;
    for (lli i = 0; i < e; i++) {
        cin >> nodes >> neighbor;
        graph[nodes].push_back(neighbor);
        graph[neighbor].push_back(nodes);
    }
    lli start;
    cin >> start;
    BFS(start, graph);
    return 0;
}

```

```

    }
    if (pattern[j] == pattern[i]) {
        pi[i] = j + 1;
    }
}
return pi;
}

void soln() {
    string text, match;
    cin >> text >> match;
    lli ct = 0;
    lli m = match.size();
    string merged = match + '#' + text;
    vector<lli> pi = pi_array(merged);
    for (lli x : pi) {
        cout << x << " ";
        if (x == m)
            ct++;
    }
    // cout << endl;
    cout << ct << endl;
}

```

Adjacency List:

```

void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> lis(n + 1);
    for (lli i = 0; i < m; i++) {
        lli u, v;
        cin >> u >> v;
        lis[u].push_back(v);
        lis[v].push_back(u);
    }
    for (lli i = 1; i <= n; i++) {
        cout << i << " -> ";
        for (auto j : lis[i]) {
            cout << j << " ";
        }
        cout << endl;
    }
}

```

Adjacency Matrix:

```

void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> mat(n + 1, vector<lli>(n + 1,
0));
    for (lli i = 0; i < m; i++) {
        lli u, v;
        cin >> u >> v;
        mat[u][v] = 1;
        mat[v][u] = 1;
    }
    for (lli i = 1; i <= n; i++) {
        for (lli j = 1; j <= n; j++) {
            cout << mat[i][j] << " ";
        }
        cout << endl;
    }
}

```

Topological Sorting:

BFS2:

```
const lli N = 1e5 + 10;
vector<bool> visited(N, false);
vector<lli> parent(N, -1);
vector<lli> dist(N, -1);
vector<vector<lli>> graph(N);
void printPath(lli node) {
    vector<lli> path;
    while (node != -1) {
        path.push_back(node);
        node = parent[node];
    }
    reverse(path.begin(), path.end());
    for (lli i = 0; i < (lli)path.size(); i++) {
        if (i > 0)
            cout << "->";
        cout << path[i];
    }
    cout << endl;
}

void BFS(lli start, vector<pair<lli, lli>> &BFSTree) {
    queue<lli> q;
    q.push(start);
    visited[start] = true;
    dist[start] = 0;
    cout << "BFS Order: ";
    while (!q.empty()) {
        lli v = q.front();
        cout << v << " ";
        q.pop();
        for (lli neighbor : graph[v]) {
            if (!visited[neighbor]) {
                q.push(neighbor);
                visited[neighbor] = true;
                parent[neighbor] = v;
                dist[neighbor] = dist[v] + 1;
                BFSTree.push_back({v, neighbor});
            }
        }
    }
    cout << endl;
}

void soln() {
    lli n, e, s;
    cin >> n >> e >> s;
    for (lli i = 1; i <= n; i++) {
        graph[i].clear();
        visited[i] = false;
        parent[i] = -1;
        dist[i] = -1;
    }
    vector<pair<lli, lli>> BFSTree;
    for (lli i = 0; i < e; i++) {
        lli u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    cout << "Adjacency List:" << endl;
    for (lli i = 1; i <= n; i++) {
        cout << i << ": ";
        for (auto j : graph[i]) {
            cout << j << " ";
        }
        cout << endl;
    }
    BFS(s, BFSTree);
    cout << "Distance from source:" << endl;
    for (lli i = 1; i <= n; i++) cout << i << "(" <<
dist[i] << ") ";
    cout << endl;
    cout << "Paths from source:" << endl;
    for (lli i = 1; i <= n; i++) {
```

```
// Using Kahn's algorithm
vector<lli> topologicalSort(vector<vector<lli>> &lis,
lli v) {
    vector<lli> inDegree(v, 0);
    for (lli i = 0; i < v; i++) {
        for (auto j : lis[i]) {
            inDegree[j]++;
        }
    }
    queue<lli> q;
    for (lli i = 0; i < v; i++) {
        if (inDegree[i] == 0) {
            q.push(i);
        }
    }
    vector<lli> ans;
    while (!q.empty()) {
        lli node = q.front();
        q.pop();
        ans.push_back(node);
        for (auto j : lis[node]) {
            inDegree[j]--;
            if (inDegree[j] == 0) {
                q.push(j);
            }
        }
    }
    if ((lli)ans.size() != v) {
        return {};
    }
    return ans;
}

void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> lis(n);
    for (lli i = 0; i < m; i++) {
        lli u, v;
        cin >> u >> v;
        lis[u].push_back(v);
    }
    vector<lli> ans = topologicalSort(lis, n);
    if (ans.empty()) {
        // Graph Contains Cycle, cannot be sorted
        cout << "[ ]" << endl;
        return;
    }
    for (auto i : ans) {
        cout << i << " ";
    }
    cout << endl;
}
```

Path Checking

```
void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> lis(n + 1);
    for (lli i = 0; i < m; i++) {
        lli u, v;
        cin >> u >> v;
        lis[u].push_back(v);
        // lis[v].push_back(u);
    }
    // for (lli i = 1; i <= n; i++) {
    //     cout << i << " -> ";
    //     for (auto j : lis[i]) {
    //         cout << j << " ";
    //     }
    //     cout << endl;
    // }
```

```

        if (dist[i] != -1){
            cout << i << ": ";
            printPath(i);
        }
    }
    cout << "Edges of BFS tree:" << endl;
    for (auto edge : BFSTree) cout << edge.first << " "
    << edge.second << endl;
}

```

DFS:

```

const lli N = 1e7 + 10;
vector<bool> visited(N);
void DFS(lli v, vector<vector<lli>>& graph) {
    visited[v] = true;
    cout << v << " ";
    for (auto next = graph[v].begin(); next !=
graph[v].end(); ++next) {
        if (!visited[*next])
            DFS(*next, graph);
    }
}
int main() {
    fast();
    lli n, e; // n = nodes, e = edges
    cin >> n >> e;
    vector<vector<lli>> graph(n + 1);
    lli nodes, neighbor;
    for (lli i = 0; i < e; i++) {
        cin >> nodes >> neighbor;
        graph[nodes].push_back(neighbor);
        graph[neighbor].push_back(nodes);
    }
    lli start;
    cin >> start;
    DFS(start, graph);
    return 0;
}

```

DFS2:

```

const lli N = 2e5 + 10;
vector<vector<lli>> graph(N);
vector<bool> visited(N, false);
vector<lli> discovery(N, 0), finishing(N, 0), parent(N,
-1);
map<pair<lli, lli>, string> edgeType;
lli tct = 0;
void classify_edge(lli u, lli v) {
    if (!visited[v]) {
        edgeType[{u, v}] = "Tree Edge";
    } else if (finishing[v] == 0) {
        edgeType[{u, v}] = "Back Edge";
    } else if (discovery[u] < discovery[v]) {
        edgeType[{u, v}] = "Forward Edge";
    } else {
        edgeType[{u, v}] = "Cross Edge";
    }
}
void DFS(lli v, vector<pair<lli, lli>>& edges) {
    visited[v] = true;
    discovery[v] = ++tct;
    cout << v << " ";
    for (lli neighbor : graph[v]) {
        classify_edge(v, neighbor);
        if (!visited[neighbor]) {
            parent[neighbor] = v;
            DFS(neighbor, edges);
        }
    }
}

```

```

lli ux, vx;
cin >> ux >> vx; // check if ux can go to vx
vector<lli> visited(n + 1, 0);
queue<lli> q;
q.push(ux);
visited[ux] = 1;
while (!q.empty()) {
    lli node = q.front();
    q.pop();
    for (auto i : lis[node]) {
        if (i == vx) {
            cout << "Yes" << endl;
            return;
        }
        if (!visited[i]) {
            visited[i] = 1;
            q.push(i);
        }
    }
}
cout << "No" << endl;
}

```

Sieve Prime:

```

const lli N = 1e7 + 10;
vector<bool> isPrime(N, true);
vector<lli> primes;
vector<lli> lp(N, 0), hp(N, 0); // Remove these if
space complexity occurs
void sievePrime() {
    isPrime[0] = false, isPrime[1] = false;
    for (lli i = 2; i <= N; i++) {
        if (isPrime[i]) {
            lp[i] = i, hp[i] = i;
            primes.push_back(i);
            for (lli j = i * 2; j <= N; j += i) {
                isPrime[j] = false;
                if (lp[j] == 0)
                    lp[j] = i;
                hp[j] = i;
            }
        }
    }
}
bool isPrimeL(lli n) {
    if (n < N)
        return isPrime[n];
    for (lli p : primes) {
        if (p * p > n)
            break;
        if (n % p == 0)
            return false;
    }
    return true;
}
void soln() {
    lli n;
    cin >> n;
    if (n > N)
        cout << (isPrimeL(n) ? "YES" : "NO") << endl;
    else
        cout << (isPrime[n] ? "YES" : "NO") << endl;
}

```

Binomial Co-efficient:

```

const lli N = 1e6 + 10;
const lli MOD = 1e9 + 7;
lli factorial[N], inverseFactorial[N], modInverse[N];
void preCalculate_nCr() {

```

```

finishing[v] = ++tct;
}
void soln() {
    lli n, e;
    cin >> n >> e;
    for (lli i = 1; i <= n; i++) {
        graph[i].clear();
        visited[i] = false;
        discovery[i] = finishing[i] = 0;
        parent[i] = -1;
    }
    edgeType.clear();
    vector<pair<lli, lli>> edges;
    for (lli i = 0; i < e; i++) {
        lli u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        edges.emplace_back(u, v);
    }
    cout << "Adjacency List:" << endl;
    for (lli i = 1; i <= n; i++) {
        cout << i << ": ";
        for (lli j : graph[i]) {
            cout << j << " ";
        }
        cout << endl;
    }
    cout << "DFS order: ";
    for (lli i = 1; i <= n; i++) {
        if (!visited[i]) {
            DFS(i, edges);
        }
    }
    cout << endl;
    cout << "Timestamps of Vertex(discovery/finishing):"
    << endl;
    for (lli i = 1; i <= n; i++) {
        cout << i << "(" << discovery[i] << "/" <<
        finishing[i] << ")" << endl;
    }
    cout << "Edge classification:" << endl;
    for (auto& [u, v] : edges) {
        cout << u << " " << v << ": " << edgeType[{u,
v}] << endl;
    }
}
}

```

Shortest Path Using BFS:

```

const lli N = 1e5 + 10;
vector<bool> visited(N, false);
vector<lli> parent(N, -1);
vector<lli> dist(N, -1);
void BFS(lli start, vector<vector<lli>>& graph) {
    queue<lli> q;
    q.push(start);
    visited[start] = true;
    dist[start] = 0;
    while (!q.empty()) {
        lli v = q.front();
        q.pop();
        for (lli neighbor : graph[v]) {
            if (!visited[neighbor]) {
                q.push(neighbor);
                visited[neighbor] = true;
                parent[neighbor] = v;
                dist[neighbor] = dist[v] + 1;
            }
        }
    }
}
void soln() {
    lli n, e;
    cin >> n >> e;

```

```

factorial[0] = 1;
for (lli i = 1; i < N; i++)
    factorial[i] = (1LL * i * factorial[i - 1] %
MOD);
modInverse[1] = 1;
for (lli i = 2; i < N; i++) {
    modInverse[i] = (-1LL * (MOD / i)) *
modInverse[MOD % i] % MOD;
    modInverse[i] = (modInverse[i] + MOD) % MOD;
}
inverseFactorial[0] = 1;
for (lli i = 1; i < N; i++)
    inverseFactorial[i] =
(1LL * modInverse[i] * inverseFactorial[i -
1] % MOD);
}
lli nCr(lli n, lli r) {
    if (n < r || n < 0 || r < 0)
        return 0;
    else
        return (1LL * factorial[n] * inverseFactorial[n
- r] % MOD *
                inverseFactorial[r] % MOD);
}
lli nPr(lli n, lli r) {
    if (n < r || n < 0 || r < 0)
        return 0;
    return (1LL * nCr(n, r) * factorial[r] % MOD);
}
void soln() {
    lli n, r;
    cin >> n >> r;
    cout << nCr(n, r) << endl;
}

```

Segment Tree:

```

const lli N = 1e7 + 10;
const lli inf = LLONG_MAX;
vector<lli> arr(N);
#define lc (2 * node)
#define rc ((2 * node) + 1)
struct SegmentTree {
    vector<lli> st;
    SegmentTree(lli n) { st.resize(4 * n, 0); }
    void build(lli node, lli b, lli e) {
        if (b == e) {
            st[node] = arr[b];
            return;
        }
        lli mid = (b + e) / 2;
        build(lc, b, mid);
        build(rc, mid + 1, e);
        st[node] = st[lc] + st[rc];
    }
    lli query(lli node, lli b, lli e, lli i, lli j) {
        if (b > j || e < i)
            return 0;
        if (b >= i && e <= j)
            return st[node];
        lli mid = (b + e) / 2;
        return query(lc, b, mid, i, j) + query(rc, mid +
1, e, i, j);
    }
    void update(lli node, lli b, lli e, lli i, lli x) {
        if (b > i || e < i)
            return;
        if (b == e && b == i) {
            st[node] = x;
            return;
        }
        lli mid = (b + e) / 2;
        update(lc, b, mid, i, x);

```

```

vector<vector<lli>> graph(n + 1);
for (lli i = 0; i < e; i++) {
    lli u, v;
    cin >> u >> v;
    graph[u].push_back(v);
    graph[v].push_back(u);
}
lli start;
cin >> start;
BFS(start, graph);
for (lli i = 1; i <= n; i++) cout << dist[i] << " ";
cout << endl;
}

```

Path Finding Grid:

```

/* A (n*m) grid where you can traverse through if there
are 0's and 1's
are blocks, your job is to find if you can reach from
top left to bottom right
*/
vector<lli> dx = {1, -1, 0, 0};
vector<lli> dy = {0, 0, 1, -1};
bool canReach(vector<vector<lli>> &grid) {
    lli n = grid.size(), m = grid[0].size();
    if (grid[0][0] == 1 || grid[n - 1][m - 1] == 1)
        return false;
    queue<pair<lli, lli>> q;
    q.push({0, 0});
    vector<vector<bool>> visited(n, vector<bool>(m,
false));
    visited[0][0] = true;
    while (!q.empty()) {
        auto [x, y] = q.front();
        q.pop();
        if (x == n - 1 && y == m - 1)
            return true;
        for (lli d = 0; d < 4; d++) {
            lli nx = x + dx[d], ny = y + dy[d];
            if (nx >= 0 && ny >= 0 && nx < n && ny < m
&& grid[nx][ny] == 0 &&
                !visited[nx][ny]) {
                visited[nx][ny] = true;
                q.push({nx, ny});
            }
        }
    }
    return false;
}
void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> grid(n, vector<lli>(m));
    for (lli i = 0; i < n; i++)
        for (lli j = 0; j < m; j++) cin >> grid[i][j];
    cout << (canReach(grid) ? "YES" : "NO") << endl;
}

```

Bipartite Graph:

```

const lli N = 1e7 + 10;
vector<bool> visited(N);
vector<lli> color(N);
bool ok;
void DFS(lli v, vector<vector<lli>>& graph) {
    visited[v] = true;
    // cout << v << " ";
    for (auto next = graph[v].begin(); next !=
graph[v].end(); ++next) {
        if (!visited[*next]) {
            color[*next] = color[v] ^ 1;

```

```

        update(rc, mid + 1, e, i, x);
        st[node] = st[lc] + st[rc];
    }
} st(N);
void soln() {
    lli n;
    cin >> n;
    for (lli i = 1; i <= n; i++) cin >> arr[i];
    st.build(1, 1, n);
    cout << "Initial Segment Tree: ";
    for (lli i = 1; i < (2 * n); i++) cout << st.st[i]
<< " ";
    cout << endl;
    lli q;
    cin >> q;
    while (q--) {
        lli x, y;
        cin >> x >> y;
        cout << "Query [" << x << ", " << y << "] -> "
<< st.query(1, 1, n, x, y) << endl;
    }
    cin >> q;
    while (q--) {
        lli i, x;
        cin >> i >> x;
        st.update(1, 1, n, i, x);
        cout << "Segment Tree after update [" << i << "]"
with " << x << ": ";
        for (lli i = 1; i < (2 * n); i++) cout <<
st.st[i] << " ";
        cout << endl;
    }
}

```

Segment Tree Lazy:

```

const lli N = 1e7 + 10;
const lli inf = LLONG_MAX;
vector<lli> arr(N);
#define lc (2 * node)
#define rc ((2 * node) + 1)
struct SegmentTree {
    vector<lli> st, lazy;
    SegmentTree(lli n) {
        st.resize(4 * n, 0);
        lazy.resize(4 * n, 0);
    }
    // Lazy propagation: Push pending updates to
children
    inline void push(lli node, lli b, lli e) {
        if (lazy[node] == 0)
            return;
        st[node] += lazy[node] * (e - b + 1);
        if (b != e) {
            lazy[lc] += lazy[node];
            lazy[rc] += lazy[node];
        }
        lazy[node] = 0;
    }
    inline lli combine(lli a, lli b) { return a + b; }
    inline void pull(lli node) { st[node] = st[lc] +
st[rc]; }
    void build(lli node, lli b, lli e) {
        lazy[node] = 0;
        if (b == e) {
            st[node] = arr[b];
            return;
        }
        lli mid = (b + e) / 2;
        build(lc, b, mid);
        build(rc, mid + 1, e);
        pull(node);
    }
    void update(lli node, lli b, lli e, lli i, lli j,

```

```

        DFS(*next, graph);
    } else {
        if (color[*next] == color[v])
            ok = false;
    }
}
}
void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> graph(n + 1);
    while (m--) {
        lli u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }
    ok = true;
    lli start;
    cin >> start;
    DFS(start, graph);
    for (lli i = 1; i <= n; i++) {
        if (!visited[i])
            DFS(i, graph); // if the graph is not
connected
    }
    if (ok)
        cout << "YES" << endl;
    else
        cout << "NO" << endl;
}

```

Cycle Detection:

```

#include <bits/stdc++.h>
using namespace std;
typedef long long int lli;
#define enl "\n"
void fast() {
    ios_base::sync_with_stdio(false);
    cin.tie(0);
}
lli const N = 2e5 + 10;
vector<lli> color(N);
vector<lli> parent(N);
bool cycle;
void DFS(lli u, vector<vector<lli>> &g) {
    color[u] = 1;
    for (auto v : g[u]) {
        if (color[v] == 0) {
            parent[v] = u;
            DFS(v, g);
        } else if (color[v] == 1) {
            cycle = true;
        }
    }
    color[u] = 2;
}
void soln() {
    lli n, m;
    cin >> n >> m;
    vector<vector<lli>> graph(n + 1);
    for (lli i = 0; i < m; i++) {
        lli u, v;
        cin >> u >> v;
        graph[u].push_back(v);
        // graph[v].push_back(u);
    }
    cycle = false;
    for (lli i = 1; i <= n; i++) {
        if (color[i] == 0)
            DFS(i, graph);
    }
    cout << (cycle ? "YES" : "NO") << enl;
}

```

```

lli v) {
    push(node, b, e);
    if (j < b || e < i)
        return;
    if (i <= b && e <= j) {
        lazy[node] = v;
        push(node, b, e);
        return;
    }
    lli mid = (b + e) / 2;
    update(lc, b, mid, i, j, v);
    update(rc, mid + 1, e, i, j, v);
    pull(node);
}
lli query(lli node, lli b, lli e, lli i, lli j) {
    push(node, b, e);
    if (i > e || b > j)
        return 0;
    if (i <= b && e <= j)
        return st[node];
    lli mid = (b + e) / 2;
    return combine(query(lc, b, mid, i, j),
query(rc, mid + 1, e, i, j));
}
} st(N);
void soln() {
    lli n;
    cin >> n;
    for (lli i = 1; i <= n; i++) cin >> arr[i];
    st.build(1, 1, n);
    cout << "Initial Segment Tree: ";
    for (lli i = 1; i < (2 * n); i++) cout << st.st[i]
<< " ";
    cout << endl;
    lli q;
    cin >> q;
    while (q--) {
        lli x, y;
        cin >> x >> y;
        cout << "Query [" << x << ", " << y << "] -> "
<< st.query(1, 1, n, x, y) << endl;
    }
    cin >> q;
    while (q--) {
        lli i, j, x;
        cin >> i >> j >> x;
        st.update(1, 1, n, i, j, x);
        cout << "Segment Tree after update [" << i << ",
" << j << "] with "
<< x << ": ";
        for (lli i = 1; i < (2 * n); i++) cout <<
st.st[i] << " ";
        cout << endl;
    }
}
}

```

Ordered Set:

```

#include <bits/stdc++.h>

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace __gnu_pbds;
using namespace std;
typedef long long int lli;
#define enl "\n"

template <typename T>
using orderedSet =
    tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
void fast() {
    ios_base::sync_with_stdio(false);
}

```

```

}
int main() {
    fast();
    lli t = 1;
    // cin >> t;
    while (t--) soln();
}
// Author: tashobi_02 //

```

Bitset:

/* The bitset in C++ STL is a container that stores a fixed-size sequence of bits (binary digits), allowing you to manipulate bits individually or as a group. It is useful for bitwise operations and is an efficient way to handle binary data when the size is known at compile time. */

```

void soln() {
    // Create a bitset of size 8 (can hold 8 bits)
    bitset<8> b1; // Default is all
    // 0s: 00000000
    bitset<8> b2(5); // Binary
    // representation of 5: 00000101
    bitset<8> b3(string("101010")); // From a string:
    // 00101010

    cout << "b1: " << b1 << endl; // Output: 00000000
    cout << "b2: " << b2 << endl; // Output: 00000101
    cout << "b3: " << b3 << endl; // Output: 00101010

    // Set, reset, and flip bits
    b1.set(1); // Set bit at index 1 to 1
    b1.set(3); // Set bit at index 3 to 1
    b2.reset(0); // Reset bit at index 0 to 0
    b3.flip(); // Flip all bits

    cout << "After modification:" << endl;
    cout << "b1: " << b1 << endl; //
    // Output: 00001010
    cout << "b2: " << b2 << endl; //
    // Output: 00000100
    cout << "b3: " << b3 << endl; //
    // Output: 11010101

    // Access and query bits
    cout << "b1[3]: " << b1[3] << endl; //
    // Output: 1 (bit at index 3)
    cout << "b1.count(): " << b1.count() << endl; //
    // Output: 2 (number of 1s)
    cout << "b3.any(): " << b3.any()
    << endl; // Output: 1 (true if any bit is 1)
    cout << "b3.none(): " << b3.none()
    << endl; // Output: 0 (false if any bit is 1)

    // Convert to string or number
    cout << "b2 as unsigned long: " << b2.to_ulong() <<
    endl; // Output: 4
    cout << "b3 as string: " << b3.to_string() << endl;
    // Output: 11010101
}

```

```

void soln() {
    /* __builtin_popcount(x) is a function that
    counts the number of set bits (1s) in the binary
    representation of an
    integer x.
    */
    lli x;
    cin >> x;
    cout << __builtin_popcountll(x) << endl;
}

```

```

cin.tie(0);
}
/*
find_by_order(k) → Returns an iterator to the k-th (0-
based) smallest element.
order_of_key(x) → Returns the count of elements strictly
smaller than x.
*/
void soln() {
    orderedSet<lli> os;
    lli n;
    cin >> n;
    for (lli i = 0; i < n; i++) {
        lli x;
        cin >> x;
        os.insert(x);
    }
    // Print elements in sorted order
    cout << "Ordered Set Elements: ";
    for (auto x : os) cout << x << " ";
    cout << endl;
    cout << "2nd smallest element: " <<
    *os.find_by_order(1) << endl;
    cout << "Elements < 5: " << os.order_of_key(5) <<
    endl;
}

```

Vector Basics:

```

void soln(){
    // Here we will see more direct and basic operations
    of vector
    // Insertions
    vector<lli> vec;
    vec.push_back(1);
    vec.push_back(2);
    vec.push_back(3);
    cout << "Vector after push_back: ";
    for (lli v : vec) cout << v << " ";
    cout << endl;
    // Resize
    vec.resize(5);
    cout << "Vector after resize to 5: ";
    for (lli v : vec) cout << v << " ";
    cout << endl;
    // Empty
    cout << "Is vector empty? " << (vec.empty() ? "Yes"
    : "No") << endl;
    // Operator[]
    cout << "Element at index 1 using operator[]: " <<
    vec[1] << endl;
    // At
    cout << "Element at index 1 using at(): " <<
    vec.at(1) << endl;
    // Front
    cout << "First element using front(): " <<
    vec.front() << endl;
    // Back
    cout << "Last element using back(): " << vec.back()
    << endl;
    // Begin
    cout << "First element using begin(): " <<
    *vec.begin() << endl;
    // End
    cout << "Element past the end using end(): " << *
    (vec.end() - 1) << endl;
    // Rbegin
    cout << "Last element using rbegin(): " <<
    *vec.rbegin() << endl;
    // Rend
    cout << "Element before the first using rend(): " <<
    *(vec.rend() - 1)
    << endl;
    // Insert
}

```



```

    cout << "Binary representation of x: " << bitset<64>
(x) << endl;
    cout << "Binary representation of x (no leading
zeros): "
        << bitset<64>(x).to_string().substr(
            bitset<64>(x).to_string().find('1'))
        << endl;
    // you need to add ll in the end for lli

/* __builtin_ctz (count trailing zeros) is a
function that counts the number of trailing zeros
in the binary
representation of an integer.
*/
    lli y;
    cin >> y;
    cout << __builtin_ctzll(y) << endl;
    cout << "Binary representation of y: " << bitset<64>
(y) << endl;
    cout << "Binary representation of y (no leading
zeros): "
        << bitset<64>(y).to_string().substr(
            bitset<64>(y).to_string().find('1'))
        << endl;
    // Compute the biggest power of 2 that is a divisor
of x. Example: f(24) = 4
    lli z;
    cin >> z;
    cout << __builtin_ctzll(z) << endl;
    cout << "Binary representation of z: " << bitset<64>
(z) << endl;
    cout << "Binary representation of z (no leading
zeros): "
        << bitset<64>(z).to_string().substr(
            bitset<64>(z).to_string().find('1'))
        << endl;
    /*
The __builtin_clz(x) is a function that counts
the number of leading zeros in the binary
representation of a number.
*/
    lli w;
    cin >> w;
    cout << __builtin_clzll(w) << endl;
    cout << "Binary representation of w: " << bitset<64>
(w) << endl;
    cout << "Binary representation of w (no leading
zeros): "
        << bitset<64>(w).to_string().substr(
            bitset<64>(w).to_string().find('1'))
        << endl;
    // Compute the smallest power of 2 that is not
smaller than x. Example:
    // f(12) = 16
    lli q;
    cin >> q;
    cout << (1 << (64 - __builtin_clzll(q - 1))) <<
endl;
    cout << "Binary representation of q: " << bitset<64>
(q) << endl;
    cout << "Binary representation of q (no leading
zeros): "
        << bitset<64>(q).to_string().substr(
            bitset<64>(q).to_string().find('1'))
        << endl;
}

```

```

vec.insert(vec.begin() + 1, 10);
cout << "Vector after insert at position 1: ";
for (lli v : vec) cout << v << " ";
cout << endl;
// Erase
vec.erase(vec.begin() + 1);
cout << "Vector after erase at position 1: ";
for (lli v : vec) cout << v << " ";
cout << endl;
// Swap
vector<lli> vec2 = {7, 8, 9};
vec.swap(vec2);
cout << "Vector after swap with vec2: ";
for (lli v : vec) cout << v << " ";
cout << endl;
// Clear
vec.clear();
cout << "Vector after clear: ";
for (lli v : vec) cout << v << " ";
cout << endl;
// Eemplace
vec.emplace(vec.begin(), 5);
cout << "Vector after emplace at begin: ";
for (lli v : vec) cout << v << " ";
cout << endl;
// Eemplace_back
vec.emplace_back(6);
cout << "Vector after emplace_back: ";
for (lli v : vec) cout << v << " ";
cout << endl;
// Assign
vec.assign(3, 4);
cout << "Vector after assign 3 elements of value 4:
";
for (lli v : vec) cout << v << " ";
cout << endl;
}

```